Graph Algorithms Pseudocode

1 Topological Sort using DFS

Time Complexity: $\mathcal{O}(|V| + |E|)$ **Space Complexity:** $\mathcal{O}(|V|)$ auxiliary

Algorithm 1 Topological Sort (DFS)

```
procedure TopologicalSortDFS(G, V)
    visited ← array of size |V| initialized to False
    S ← empty stack
    for each vertex u ∈ V do
        if not visited[u] then
            DFS-Topo(u, visited, S)
        end if
    end for
    while S is not empty do
        print Pop(S)
    end while
end procedure

procedure DFS-Topo(u, visited, S)
    visited[u] ← True
    for each neighbor v of u do
        if not visited[v] then
            DFS-Topo(v, visited, S)
        end if
    end for
    Push(S, u)
end procedure
```

2 Topological Sort using BFS (Kahn's Algorithm)

Time Complexity: $\mathcal{O}(|V| + |E|)$ **Space Complexity:** $\mathcal{O}(|V|)$ auxiliary

Algorithm 2 Topological Sort (BFS)

```
procedure TopologicalSortBFS(G, V)
    in_degree ← array of size |V| initialized to 0
    for each u ∈ V do
        for each neighbor v of u do
            in_degree[v] ← in_degree[v] + 1
        end for
    end for
    Q ← empty queue
    for each vertex u ∈ V do
        if in_degree[u] = 0 then
            Enqueue(Q, u)
        end if
    end for
    while Q is not empty do
        u ← Dequeue(Q)
        print u
        for each neighbor v of u do
            in_degree[v] ← in_degree[v] − 1
            if in_degree[v] = 0 then
                Enqueue(Q, v)
            end if
        end for
    end while
end procedure
```

3 DFS Code for Multiple Components

Time Complexity: $\mathcal{O}(|V| + |E|)$ **Space Complexity:** $\mathcal{O}(|V|)$ auxiliary

Algorithm 3 DFS for Multiple Components

```
1: procedure DFS-MultiComponent(G, V)
2:     visited ← array of size |V| initialized to False
3:     components ← 0
4:     for each vertex u ∈ V do
5:         if not visited[u] then
6:             components ← components + 1
7:             DFS-Visit(u, visited)
8:         end if
9:     end for
10:    return components
11: end procedure

12: procedure DFS-Visit(u, visited)
13:    visited[u] ← True
14:    for each neighbor v of u do
15:        if not visited[v] then
16:            DFS-Visit(v, visited)
17:        end if
18:    end for
19: end procedure
```

4 Kosaraju's Algorithm for Strongly Connected Components

Time Complexity: $\mathcal{O}(|V| + |E|)$ **Space Complexity:** $\mathcal{O}(|V|)$ auxiliary

Algorithm 4 Kosaraju's Algorithm

```
1: procedure KOSARAJU(G, V)
2:     visited ← array of size |V| initialized to False
3:     S ← empty stack
                                                    ▷ Pass 1: Record finishing times
4:     for each vertex u ∈ V do
5:         if not visited[u] then
6:             FILLORDER(u, visited, S)
7:         end if
8:     end for
9:     G^T ← Transpose of G                          ▷ Reverse all edge directions
10:    visited ← array of size |V| initialized to False
                                    ▷ Pass 2: DFS on transposed graph in order of finishing times
11:    while S is not empty do
12:        u ← POP(S)
13:        if not visited[u] then
14:            DFS-SCC(G^T, u, visited)
15:            print "— End of SCC —"
16:        end if
17:    end while
18: end procedure

19: procedure FILLORDER(u, visited, S)
20:    visited[u] ← True
21:    for each neighbor v of u do
22:        if not visited[v] then
23:            FILLORDER(v, visited, S)
24:        end if
25:    end for
26:    PUSH(S, u)
27: end procedure

28: procedure DFS-SCC(G^T, u, visited)
29:    visited[u] ← True
30:    print u
31:    for each neighbor v of u in G^T do
32:        if not visited[v] then
33:            DFS-SCC(G^T, v, visited)
34:        end if
35:    end for
36: end procedure
```

5 Dijkstra's Algorithm (Single-Source Shortest Path)

Time Complexity: $\mathcal{O}((|V| + |E|) \log |V|)$ **Space Complexity:** $\mathcal{O}(|V|)$ auxiliary

Algorithm 5 Dijkstra's Algorithm

```
1: procedure DIJKSTRA(G, V, w, s)
2:     for each vertex v ∈ V do
3:         dist[v] ← ∞
4:         prev[v] ← null
5:     end for
6:     dist[s] ← 0
7:     Q ← priority queue containing all v ∈ V keyed by dist
8:     while Q is not empty do
9:         u ← EXTRACTMIN(Q)
10:        for each neighbor v of u do
11:            alt ← dist[u] + w(u, v)
12:            if alt < dist[v] then
13:                dist[v] ← alt
14:                prev[v] ← u
15:                DECREASEKEY(Q, v, alt)
16:            end if
17:        end for
18:    end while
19: end procedure
```

6 Kruskal's Algorithm (Minimum Spanning Tree)

Time Complexity: $\mathcal{O}(|E| \log |E|)$ **Space Complexity:** $\mathcal{O}(|V|)$ auxiliary (for Union-Find)

Algorithm 6 Kruskal's Algorithm

```
1: procedure KRUSKAL(G, V, E, w)
2:     A ← ∅
3:     for each vertex v ∈ V do
4:         MAKESET(v)
5:     end for
6:     Sort the edges of E into non-decreasing order by weight w
7:     for each edge (u, v) ∈ E taken in sorted order do
8:         if FINDSET(u) ≠ FINDSET(v) then
9:             A ← A ∪ {(u, v)}
10:            UNION(u, v)
11:        end if
12:    end for
13:    return A
14: end procedure
```

7 Prim's Algorithm (Minimum Spanning Tree)

Time Complexity: $\mathcal{O}((|V| + |E|) \log |V|)$ **Space Complexity:** $\mathcal{O}(|V|)$ auxiliary

Algorithm 7 Prim's Algorithm

```
1:  procedure PRIM(G, V, w, r)
2:      for each u ∈ V do
3:          key[u] ← ∞
4:          prev[u] ← null
5:      end for
6:      key[r] ← 0
7:      Q ← priority queue containing all v ∈ V keyed by key
8:      while Q is not empty do
9:          u ← EXTRACTMIN(Q)
10:         for each neighbor v of u do
11:             if v ∈ Q and w(u, v) < key[v] then
12:                 prev[v] ← u
13:                 key[v] ← w(u, v)
14:                 DECREASEKEY(Q, v, key[v])
15:             end if
16:         end for
17:     end while
18: end procedure
```